

Une tâche de classification galactique et ses améliorations possibles.

Camille Dussauchoy et Delfin Ergelik

Projet final pour le cours d'Applications du Deep Learning, Mai 2026

Table des matières

1	Introduction	3
1.1	Quelle est la motivation pour cette tâche de classification galactique?	3
1.2	Présentation des données manipulées et de leur interprétation scientifique.	3
2	La base de données	4
2.1	Le pré-traitement du dataset	4
2.2	Modifications du dataset, et la <i>data augmentation</i>	5
3	L'approche de machine learning.	6
3.1	Les principales méthodes	6
3.2	Les résultats de machine learning	6
3.2.1	Arbre de décision	6
3.2.2	Random Forest	7
4	Construction des modèles de Deep learning	8
4.1	Les architectures	8
4.1.1	Premier modèle : Petit réseau convolutionnel	8
4.1.2	Deuxième modèle : Variational Autoencoder (VAE)	9
4.1.3	Troisième modèle : ResNet-18	9
4.1.4	Quatrième modèle : ResNet-34	10
4.2	L'entraînement	10
4.3	Les résultats et l'évaluation des modèles	12
4.3.1	Petit réseau convolutionnel	12
4.3.2	Premier modèle avec l'aide d'un VAE	13
4.3.3	Troisième modèle, Residual Network (ResNet-18)	15
4.3.4	Quatrième modèle, Residual Network (ResNet-34)	15
4.3.5	Synthèse	16
5	Conclusion	17

1 Introduction

1.1 Quelle est la motivation pour cette tâche de classification galactique ?

Les tâches de classification sont très utiles dans différents domaines. Au cours de ce projet, nous allons nous plonger dans le domaine de l'Astronomie pour démontrer l'utilité des réseaux de neurones et d'autres techniques pour la classification d'objets galactiques. En astronomie, l'enjeu est souvent, à partir de données comme des images, de réaliser un certain nombre de traitements dans le but de distinguer des propriétés caractéristiques des objets définis et étudiés, ou encore de classer les différents objets possiblement identifiables dans un champ donné. Plus précisément, notre objectif est le suivant : à partir d'un dataset pré-traité, entraîner un modèle afin d'effectuer une classification efficace des galaxies des systèmes qui entourent le système solaire. Il est également intéressant de pratiquer une classification avec de l'apprentissage statistique (machine learning), autrement dit, sans utiliser de réseaux de neurones, et nous explorerons ces possibilités afin de comparer raisonnablement les résultats et l'efficacité des diverses méthodes utilisées.

En astronomie, la majorité des problèmes nécessitent de larges connaissances en science des données et en traitement de l'image et du signal. En effet, des missions comme la mission DESI (Dark Energy Spectroscopic Instrument), dont le but est d'étudier la nature de l'énergie sombre, montrent le rôle important du traitement raisonné des données dans l'optique de générer des résultats optimaux, dans le cas de cette mission, les chercheuses et chercheurs construisent une carte tridimensionnelle résumée de la situation de l'énergie sombre dans le temps. La mission que nous proposons à travers ce rapport est bien plus modeste, cela va de soi, mais elle permet de revenir à l'origine des processus tant utilisés aujourd'hui, pour des missions qui pourraient changer notre conception de l'univers.

Quelques questions fondamentales doivent rester au centre de notre développement, en amont des essais techniques à entreprendre. Il s'agit de se rappeler les principes simples : comment optimiser la réalisation de la tâche en permettant un impact moindre pour l'environnement (est-il nécessaire d'utiliser des LLM ou des objets plus complexes) ? Le dataset utilisé convient-il à la tâche que nous souhaitons entreprendre (la taille, la qualité des images...) ? Quelles améliorations de ce dataset faudrait-il envisager alors ? Comment définir un modèle et l'optimiser (méthode de back-propagation) ? Comment qualifier la performance de notre modèle ? Quels changements y apporter ?

Nous allons répondre à ces questions, en suivant le développement ci-contre : Premièrement, nous allons présenter le dataset ainsi que les améliorations et les changements apportés. Deuxièmement, nous allons tester les méthodes de machine learning afin d'avoir un seuil en termes de performance du modèle. Troisièmement, nous construirons différents modèles en testant leur efficacité. Enfin, nous conclurons sur la méthode la plus adaptée pour résoudre le problème de classification posé.

1.2 Présentation des données manipulées et de leur interprétation scientifique.

Les données utilisées proviennent du même site mais sont manipulées et mises en forme de différentes manières. Nous avons, d'une part, le tableau dans lequel sont répertoriés les types de galaxies, et d'autre part, nous avons des images de galaxies, que nous allons améliorer et traiter pour poursuivre le projet. Dans la table téléchargée [1], les différents noms des colonnes sont les suivants :

OBJID, RA, DEC, NVOTE, PEL, PCW, PACW, PEDGE, PDK, PMG, PCS, PELDEBIASED, PCSDEBIASED, SPIRAL, ELLIPTICAL, UNCERTAIN. Les quatre premières colonnes désignent respectivement l'identifiant unique de l'objet, les coordonnées célestes (Ascension droite et Déclinaison), nombre de votes pour la classification par les participants. De plus, toutes les colonnes commençant par P représentent les probabilités données par les participants pour déterminer le type de la galaxie observée.

Nous trouvons les galaxies spirales (SPIRAL) qui sont des disques aplatis avec des bras en spirale. Elles contiennent beaucoup de gaz et de poussière et une formation d'étoiles active. Les sous-types de ce premier type sont indiqués par : PCW (spirale tournant dans le sens horaire) et PACW (spirale tournant dans le sens antihoraire). Il se trouve également des galaxies elliptiques (ELLIPTICAL ou PEL) qui sont des formes sphériques ou ellipsoïdales, avec peu de gaz et de poussière et une population d'étoiles âgées, avec une formation de nouvelles étoiles très peu active. Il y a aussi des galaxies vues par la tranche (PEDGE) qui sont des galaxies souvent spirales, observées de profil, et qui ont une structure interne difficile à distinguer. Les galaxies en fusion (PMG) sont des systèmes en interaction ou en collision, qui présentent une morphologie perturbée et une formation d'étoiles très probable. Les galaxies compactes ou symétriques (PCS) sont des objets compacts parfois sphériques dont la structure est majoritairement peu visible. Le dernier type regroupe les cas incertains (UNCERTAIN ou PDK), dont la classification est ambiguë ou difficile, dans le cas de données insuffisantes ou d'image peu claire.

Le processus de classification préalable par les chercheuses et chercheurs, suit le schéma ci-dessus [4]. Ils analysent d'abord la texture, puis la forme générale (disons à grande échelle), pour se concentrer sur les détails, comme les formes plus spécifiques qui complètent la forme déjà trouvée. Des autres questions comme des propriétés supplémentaires (par exemple un anneau, des irrégularités, de la poussière ou d'autres éléments) sont demandées pour faire en sorte de connaître la catégorie correspondante.

Concernant le dataset qui contient les images, nous avons d'abord recueilli sur le site Galaxy Zoo 2 [2], un dataset qui contient 14 060 images différentes de galaxies. La qualité des images peut être améliorée mais ce sont des images en couleurs (trois canaux RGB). Nous avons ensuite reconsidéré l'utilisation de ce dataset, suite à des problèmes techniques rencontrés.

2 La base de données

2.1 Le pré-traitement du dataset

La première étape du traitement des données est d'établir le lien entre le tableau de classification téléchargé en format csv et le document contenant les images. Nous allons procéder en utilisant les identifiants des images, qui correspondent à une ligne dans le tableau csv donné. Ainsi, il est aisé de créer un ensemble de données labélisées afin de procéder à une classification. Étant donné que les deux fichiers ne présentaient pas de correspondance évidente, nous avons décidé de créer notre propre dataset, en sélectionnant des images de galaxies parmi les quelques 600 000 images proposées sur le site de Galaxy Zoo.

Pour ce faire, nous avons défini une fonction qui permet de récupérer depuis un lien externe, les images correspondantes à certaines lignes du fichier csv donné en input. De cette manière, nous obtenons un dataset équilibré de 15 000 images, chaque classe (elliptical, spiral, uncertain) est représentée par 5 000 images. De plus, cette fonction permet de créer un "manifeste", qui est un fichier csv, qui

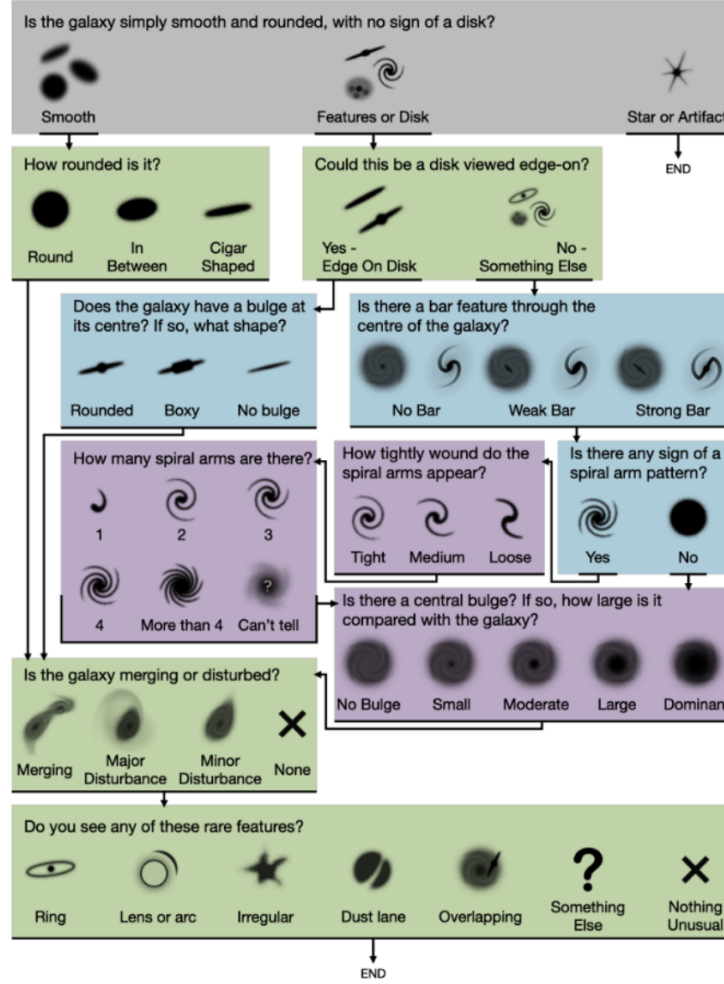


FIGURE 1 – Decision tree used by GZD-5 (Walmsley et al. 2022a). This shows the order in which questions are shown to volunteers based on their answers to previous questions.

regroupe en un tableau, les différentes caractéristiques des galaxies étudiées, que les images représentent. Une autre fichier csv répertorie les erreurs lors de la génération. Il signale, dans notre cas, 18 erreurs techniques, ce qui signifie que 18 images ont été omises dans le dataset. Nous pouvons considérer que ces erreurs ne vont pas impacter le reste du projet étant donné leur impact marginal très faible sur l'ensemble du dataset. Ensuite, il nous faut vérifier que les images ont globalement été bien sélectionnées afin de s'assurer que notre classification sera pertinente. Seulement après cette vérification, nous pouvons commencer le data augmentation et la réelle manipulation de ces données.

2.2 Modifications du dataset, et la *data augmentation*

L'étape suivante est le traitement de ce nouveau dataset, notamment l'amélioration de ces images, en terme de contraste et de luminosité (point de vue purement qualitatif) et en terme de quantité aussi, pour avoir plus de représentation de ces images. Nous considérons alors la rotation, le pivot, un changement d'échelle (zoom), afin d'augmenter le dataset. Ces idées sont générales, et nous les

avons retrouvées dans le papier de Ashley Spindler et Leon H. Butterworth, 2026 [4], dans lequel ils expliquent que l’augmentation des données permet d’améliorer les modèles de machine learning en astronomy en général, donc pour la tâche à laquelle nous nous appliquons également.

3 L’approche de machine learning.

3.1 Les principales méthodes

Avant même de procéder à l’augmentation des données, nous avons décidé d’effectuer la tâche avec des modèles de machine learning très efficaces pour la classification : les arbres de décisions et les forêts aléatoires. Ces méthodes sont classiques en machine learning [3] puisqu’elles fonctionnent comme une classification à partir de probabilités, dont découle une arborescence qui mène à la sélection d’une telle classe ou d’une telle autre classe. Etant donné que nous voulions nous concentrer sur les modèles de Deep Learning, il nous a semblé suffisant de réaliser ces deux classifications, en termes d’outils de machine learning.

3.2 Les résultats de machine learning

3.2.1 Arbre de décision

Le premier modèle testé est un arbre de décision appliqué directement aux images de galaxies. Le dataset utilisé contient au total 14 982 images réparties de manière équilibrée entre les trois classes étudiées : 4 994 galaxies spirales, 4 995 galaxies elliptiques et 4 993 galaxies incertaines. Afin d’évaluer correctement les capacités de généralisation du modèle, nous avons réalisé une séparation stratifiée des données avec 80 % des images utilisées pour l’entraînement et 20 % pour le test, soit 11 985 images d’entraînement et 2 997 images de test.

Les résultats montrent tout d’abord une accuracy d’entraînement très élevée, égale à 95.43 %, En revanche, l’accuracy obtenue sur l’ensemble de test est à 55.79 %, Cette différence importante entre les performances d’entraînement et de test met en évidence un phénomène d’overfitting. Le modèle mémorise très bien les données d’apprentissage mais parvient moins à généraliser correctement sur de nouvelles images. Ce comportement est fréquent avec les arbres de décision appliqués aux images, car une image contient un très grand nombre de variables correspondant aux pixels. Le modèle traite alors chaque pixel indépendamment et apprend des règles très spécifiques aux données d’entraînement. Il mémorise donc facilement certains détails des images sans réellement apprendre des caractéristiques générales des galaxies.

Les résultats sont tout de même supérieurs au ‘hasard’. Avec trois classes équilibrées, une classification aléatoire produirait une accuracy proche de 33 %, tandis que notre modèle atteint près de 56 %, ce qui montre qu’il parvient malgré tout à extraire certaines caractéristiques pertinentes des galaxies.

L’analyse détaillée des performances par classe montre également des différences importantes selon les catégories étudiées. Les galaxies elliptiques sont les mieux reconnues par le modèle, avec un score F1 de 0.65, une précision de 0.63 et un rappel de 0.67. Cela suggère que leur structure visuelle est relativement simple et homogène, ce qui facilite leur identification. Les galaxies spirales obtiennent également des résultats satisfaisants avec un score F1 de 0.59, une précision de 0.60 et un rappel de 0.59. En revanche, la classe incertain est relativement plus difficile à classer. Le score F1 n’atteint que 0.42, avec une précision de 0.44 et un rappel de 0.41. Cette difficulté était attendue puisque cette catégorie regroupe précisément les images difficilement interprétables. Les

galaxies incertaines possèdent souvent des caractéristiques intermédiaires ou des images de faible qualité, ce qui rend leur distinction beaucoup plus complexe.

La matrice de confusion confirme cette idée. Une partie importante des galaxies uncertain est confondue avec les classes spiral ou elliptical. À l'inverse, les galaxies elliptiques présentent des prédictions beaucoup plus stables.

Plusieurs améliorations pourraient être envisagées afin d'augmenter les performances du modèle. Tout d'abord, il serait possible d'extraire de meilleures caractéristiques visuelles avant la classification, par exemple à l'aide de techniques de détection de contours ou d'analyse de texture. Ensuite, l'utilisation de méthodes d'ensemble comme les forêts aléatoires permettrait de réduire l'overfitting et d'améliorer la robustesse des prédictions. Enfin, des techniques de pré-traitement plus avancées ou l'utilisation de modèles de deep learning, notamment les réseaux de neurones convolutionnels, semblent être une bonne idée pour bien exploiter l'information contenue dans les images galactiques.

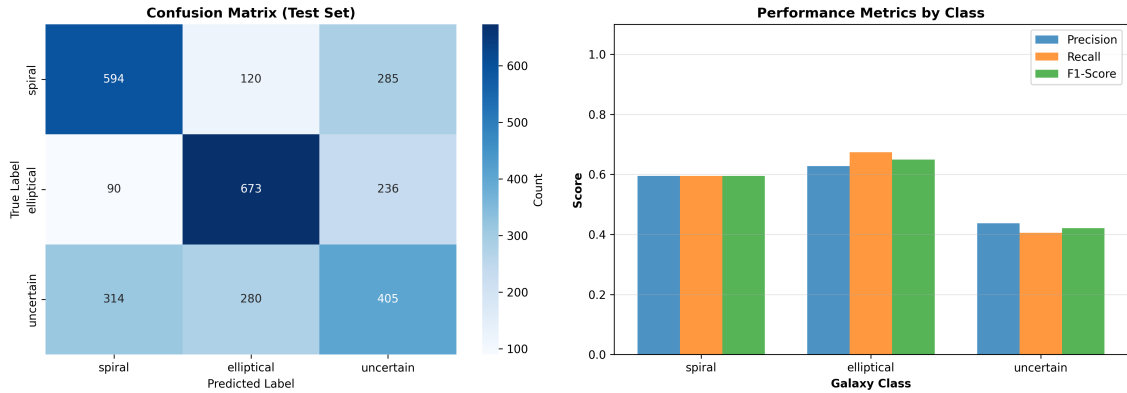


FIGURE 2 – Résultats pour les decision trees

3.2.2 Random Forest

Le second modèle de machine learning testé est une forêt aléatoire. Contrairement à l'arbre de décision simple, cette méthode repose sur un ensemble de plusieurs arbres entraînés sur différentes parties des données. Les prédictions finales sont obtenues par vote majoritaire entre les arbres, ce qui permettrait de réduire l'overfitting et d'améliorer la capacité de généralisation du modèle.

Les résultats obtenus montrent une amélioration notable par rapport au modèle précédent. La précision globale atteint environ 64.90 % sur l'ensemble de test, ce qui représente une progression importante par rapport aux 55.79 % obtenus avec l'arbre de décision unique.

L'analyse de la matrice de confusion permet d'étudier plus précisément le comportement du modèle. Pour les galaxies spirales, 707 images sont correctement classifiées dans la bonne catégorie. Toutefois, 85 images sont confondues avec des galaxies elliptiques et 207 sont classifiées comme uncertain. Cela montre que le modèle reconnaît relativement bien les structures spirales, même si certaines images ambiguës restent difficiles à distinguer.

Les galaxies elliptiques constituent la catégorie la mieux reconnue par le modèle. En effet, 742 images sont correctement prédites comme elliptical, tandis que seulement 29 sont confondues avec spiral et 228 avec uncertain. Ce résultat confirme que les galaxies elliptiques possèdent des caractéristiques visuelles relativement homogènes, ce qui facilite leur apprentissage par le modèle.

Concernant la classe uncertain, les résultats restent plus limités. Le modèle classe correctement 496 images dans cette catégorie, mais 286 sont prédites comme spiral et 217 comme elliptical. Cette

difficulté était attendue car les galaxies incertaines présentent souvent des caractéristiques intermédiaires entre plusieurs classes ou même des images de qualité insuffisante pour permettre une distinction nette. Le graphique des performances par classe confirme ces observations. La classe elliptical obtient les meilleurs scores sur l'ensemble des métriques : précision, rappel et score F1. Les galaxies spiral présentent également de très bonnes performances, avec des valeurs proches de celles des galaxies elliptiques. En revanche, la classe uncertain reste la plus faible. Les scores sont nettement plus bas, avec une précision proche de 0.52, un rappel autour de 0.50 et un score F1 de 0.51.

L'analyse des métriques montre également que les performances sont plus équilibrées que dans le cas de l'arbre de décision simple. Les écarts entre précision et rappel restent relativement faibles pour les classes spiral et elliptical, ce qui signifie que le modèle est globalement stable dans ses prédictions. Les courbes indiquent aussi une amélioration générale de la reconnaissance des galaxies par rapport au premier modèle.

Ces résultats montrent que la forêt aléatoire parvient mieux à distinguer les galaxies clairement spiral et elliptical grâce à l'agrégation des prédictions de plusieurs arbres. Cependant, la classe uncertain demeure problématique. Une grande partie des galaxies incertaines est encore prédite comme spiral ou elliptical, ce qui signifie que le modèle ne dispose pas encore de caractéristiques suffisamment discriminantes pour identifier correctement cette catégorie.

Même si les résultats restent imparfaits, cette approche constitue une amélioration significative par rapport à l'arbre de décision classique. Elle confirme également l'intérêt des méthodes plus avancées pour le traitement d'images complexes. Néanmoins, les limites observées suggèrent que les modèles de deep learning seraient probablement mieux adaptés à cette tâche de classification galactique.

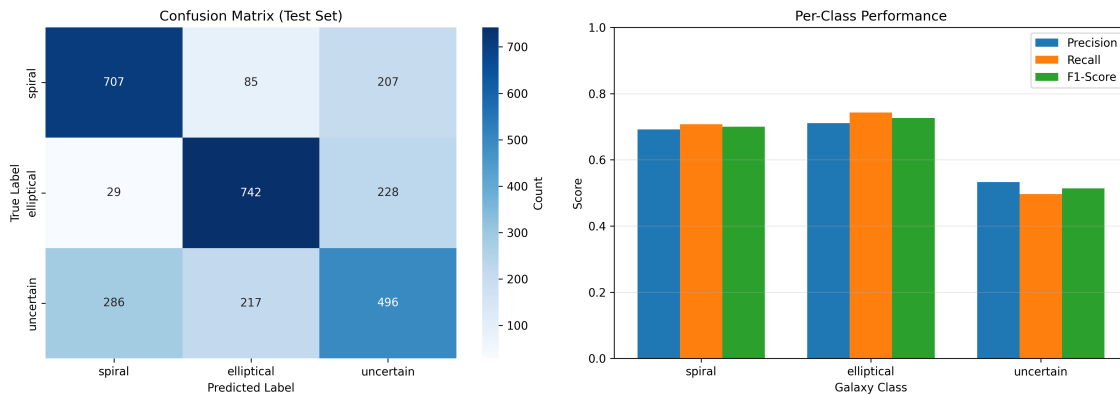


FIGURE 3 – Résultats pour les Random Forests.

4 Construction des modèles de Deep learning

4.1 Les architectures

4.1.1 Premier modèle : Petit réseau convolutionnel

Le premier modèle que nous considérons est plutôt simple. Il s'agit d'un réseau de neurones convolutionnel, c'est-à-dire qu'il présente une couche de convolution, élément clé dans le traitement d'images puisqu'elle garantit la reconnaissance de caractéristiques précises des images. En effet, les réseaux convolutionnels sont particulièrement adaptés au traitement d'images, car ils permettent

d’extraire automatiquement des caractéristiques visuelles importantes telles que les contours, les textures, les formes ou encore certaines structures locales présentes dans les images.

TABLE 1 – Architecture du réseau de neurones

Numéro de couche	Type de couche	Dimensions	Nom (PyTorch)
1	Convolution	(3, 6, 5)	<code>nn.Conv2d</code>
2	Activation	–	<code>torch.relu</code>
3	Pooling	(taille _{input} , 2)	<code>nn.functional.max_pool2d</code>
4	Fully Connected	(62 × 62 × 6, 3)	<code>nn.Linear</code>

4.1.2 Deuxième modèle : Variational Autoencoder (VAE)

Le second modèle que nous construisons se base sur la notion d’encodage de l’information, dans un espace latent, puis du décodage de l’information, afin d’optimiser la qualité des images, par exemple en essayant le débruitage des images. Un modèle U-Net variational autoencoder est idéal pour mener cette tâche à bien. Contrairement à un réseau de classification classique, le VAE n’a pas uniquement pour objectif de prédire une classe, mais également de reconstruire les images d’entrée après les avoir compressées dans une représentation de plus faible dimension.

Nous nous inspirons notamment des travaux de S. S. Mirzoyan [5] sur un autre dataset dans un papier de juin 2025 et notamment de ses idées par rapport à l’architecture du modèle. Cependant, nous avons réduit le nombre de couches de l’encoder et pour le décodeur étant donné la complexité de l’entraînement qui résulterait d’une architecture plus avancée encore que celle que nous proposons. Néanmoins, le troisième modèle a pour but de rectifier les défauts du VAE. Le but est donc d’apprendre une bonne représentation des données, puis d’entraîner un classificateur simple en récupérant la dimension de l’espace latent. Cette méthode est peu commune pour la classification, nous essayons d’utiliser les objets présentés en cours dans un objectif de diversification de leurs usages et d’expérimentation, sur un dataset a priori peu complexes (mise à part la qualité des images).

TABLE 2 – Architecture de l’encodeur du VAE

Numéro de couche	Type de couche	Dimensions	Nom (PyTorch)
1	Convolution	(3, 32, 3)	<code>nn.Conv2d</code>
2	Activation	–	<code>nn.ReLU</code>
3	Convolution	(32, 64, 3)	<code>nn.Conv2d</code>
4	Activation	–	<code>nn.ReLU</code>
5	Convolution	(64, 128, 3)	<code>nn.Conv2d</code>
6	Activation	–	<code>nn.ReLU</code>
7	Flatten	(128 × 16 × 16)	<code>view()</code>
8	Fully Connected (mean)	(32768, 64)	<code>nn.Linear</code>
9	Fully Connected (logvar)	(32768, 64)	<code>nn.Linear</code>

4.1.3 Troisième modèle : ResNet-18

Le troisième modèle est un réseau résiduel. Ce type de modèle, introduit en 2015 sur le dataset d’application ImageNet, expose un principe très simple : à cause de la rétropropagation du gradient et la perte de l’information à chaque couche, des problèmes se forment durant l’entraînement lorsque

TABLE 3 – Architecture du décodeur du VAE

Numéro de couche	Type de couche	Dimensions	Nom (PyTorch)
1	Fully Connected	(64, 32768)	<code>nn.Linear</code>
2	Reshape	(128, 16, 16)	<code>view()</code>
3	Transposed Convolution	(128, 64, 4)	<code>nn.ConvTranspose2d</code>
4	Activation	–	<code>nn.ReLU</code>
5	Transposed Convolution	(64, 32, 4)	<code>nn.ConvTranspose2d</code>
6	Activation	–	<code>nn.ReLU</code>
7	Transposed Convolution	(32, 3, 4)	<code>nn.ConvTranspose2d</code>
8	Activation de sortie	–	<code>nn.Sigmoid</code>

les couches du réseau augmentent. Alors, l'idée est d'utiliser des connexions résiduelles entre les couches pour garder l'information. Etant donné la capacité de stockage importante nécessaire au maintien d'un niveau d'information du début à la fin, cela implique que la complexité de ResNet a été davantage étudiée. Nous nous inspirons de l'idée de Han et al. [6] afin de mettre en place le ResNet, bien que l'architecture soit différente. Dans le papier, ils utilisent un ResNet-50 tandis que nous utilisons un modèle plus modeste, le ResNet-18.

TABLE 4 – Architecture de ResNet-18

Numéro	Type de couche	Dimensions	Nom (PyTorch)
1	Convolution	(3, 64, 7×7)	<code>nn.Conv2d</code>
2	Batch Normalization	(64)	<code>nn.BatchNorm2d</code>
3	Activation (ReLU)	–	<code>nn.ReLU</code>
4	Max Pooling	(3×3 , <i>stride</i> = 2)	<code>nn.MaxPool2d</code>
5	Residual Block $\times 2$	(64, 64, 3×3)	<code>BasicBlock</code>
6	Residual Block $\times 2$	(64, 128, 3×3)	<code>BasicBlock</code>
7	Residual Block $\times 2$	(128, 256, 3×3)	<code>BasicBlock</code>
8	Residual Block $\times 2$	(256, 512, 3×3)	<code>BasicBlock</code>
9	Average Pooling	(1×1)	<code>nn.AdaptiveAvgPool2d</code>
10	Fully Connected	(512, <code>num_classes</code>)	<code>nn.Linear</code>

4.1.4 Quatrième modèle : ResNet-34

Enfin, le quatrième modèle est également un réseau résiduel, seulement nous avons augmenté le nombre de couches utiles pour l'apprentissage, passant de 18 à 34 couches au total. L'architecture est donc légèrement modifiée et amplifie la complexité du modèle précédent : Comme pour le modèle précédent, l'architecture débute par une couche de convolution suivie d'une Batch Normalization et d'une activation ReLU. Une couche de max pooling réduit ensuite les dimensions spatiales afin de diminuer le coût computationnel. Le réseau est ensuite organisé en plusieurs groupes de blocs résiduels. À chaque étape, le nombre de canaux augmente progressivement tandis que la résolution spatiale diminue.

4.2 L'entraînement

TABLE 5 – Architecture de ResNet-34

Numéro	Type de couche	Dimensions	Nom (PyTorch)
1	Convolution	$(3, 64, 7 \times 7)$	<code>nn.Conv2d</code>
2	Batch Normalization	(64)	<code>nn.BatchNorm2d</code>
3	Activation (ReLU)	–	<code>nn.ReLU</code>
4	Max Pooling	$(3 \times 3, \text{stride} = 2)$	<code>nn.MaxPool2d</code>
5	Residual Block $\times 3$	$(64, 64, 3 \times 3)$	<code>BasicBlock</code>
6	Residual Block $\times 4$	$(64, 128, 3 \times 3)$	<code>BasicBlock</code>
7	Residual Block $\times 6$	$(128, 256, 3 \times 3)$	<code>BasicBlock</code>
8	Residual Block $\times 3$	$(256, 512, 3 \times 3)$	<code>BasicBlock</code>
9	Average Pooling	(1×1)	<code>nn.AdaptiveAvgPool2d</code>
10	Fully Connected	$(512, \text{num_classes})$	<code>nn.Linear</code>

Pour chacun des modèles décrits ci-dessus, nous effectuons un entraînement sur un maximum de 50 epochs (étant donné que seul le CPU est disponible, il s’agit d’optimiser le temps d’exécution des entraînements des modèles). Dans un premier temps, nous entraînons les modèles sur une vingtaine d’epochs. Si les résultats suggèrent une amélioration possible de l’apprentissage, nous augmentons ensuite le nombre d’epochs. Pour les modèles de classification, l’entraînement est réalisé avec la fonction de perte Cross-Entropy. Cette fonction est particulièrement adaptée à la classification, car elle compare la classe prédite par le modèle avec la vraie classe de l’image. Plus la probabilité attribuée à la bonne classe est élevée, plus faible est la perte. Pour le VAE, nous utilisons une fonction de perte différente, composée de deux termes : la Binary Cross-Entropy et la divergence de Kullback-Leibler [5] : La Binary Cross-Entropy mesure l’écart entre l’image d’origine et l’image reconstruite par le décodeur :

$$\mathcal{L}_{\text{BCE}}(x, \hat{x}) = - \sum_{i=1}^n [x_i \log(\hat{x}_i) + (1 - x_i) \log(1 - \hat{x}_i)] \quad (1)$$

où x représente la vraie distribution des données, $\hat{x}$ représente la distribution des données prédite par le décodeur, et n est le nombre de pixels.

La divergence de Kullback-Leibler, qui mesure la distance entre la distribution du prior et du posterior, est définie comme suit :

$$\mathcal{L}_{\text{KL}} = -\frac{1}{2} \sum_{j=1}^d (1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2) \quad (2)$$

où μ et σ sont la moyenne et la variance des vecteurs des variables latentes, et d est la dimension de l’espace latent.

La perte totale du VAE s’écrit donc :

$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{BCE}} + \beta \mathcal{L}_{\text{KL}} \quad (3)$$

En utilisant la Cross-Entropy, qui dépend de la probabilité à la classe correcte, et qui est d’autant plus faible que cette probabilité est élevée, nous nous assurons que le modèle est bien entraîné. Il faudra faire attention à un détail : cette perte ne prend que des valeurs entre 0 et 1 donc il ne faudra

pas mettre de normalisation pour les transformations apportées au dataset. L'optimiseur choisi pour l'entraînement est Adam, car il permet d'adapter automatiquement le taux d'apprentissage de chaque paramètre et donne généralement de bons résultats pour l'entraînement des réseaux de neurones.

4.3 Les résultats et l'évaluation des modèles

4.3.1 Petit réseau convolutionnel

Le premier modèle a été entraîné sur 50 epochs. L'évolution de la loss et de l'accuracy d'entraînement permet d'observer le comportement du réseau au cours de l'apprentissage. Les graphiques ci-dessous présentent respectivement l'évolution de la fonction de perte (training loss) et de l'accuracy d'entraînement au fil des epochs. Le modèle présente une erreur d'entraînement relativement faible et un comportement globalement conforme à nos attentes.

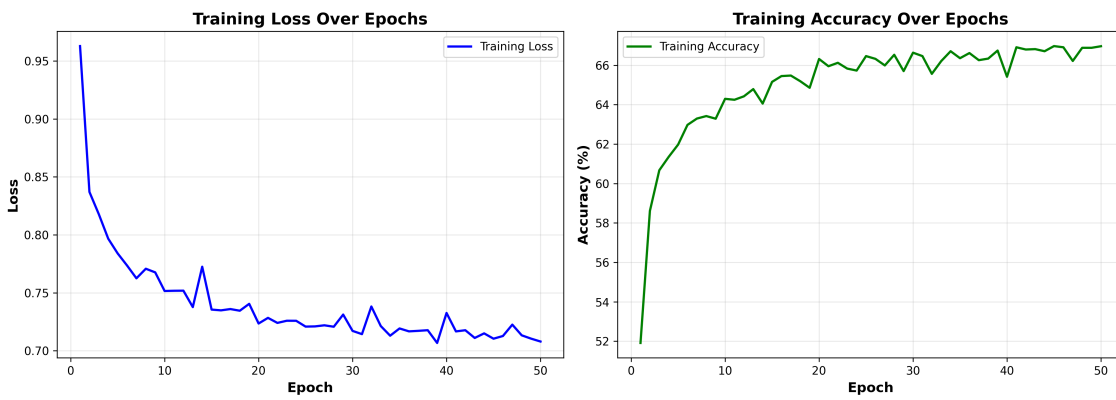


FIGURE 4 – Graphique, entraînement du petit CNN (modèle 1).

Sur le premier graphique, nous observons une diminution rapide de la loss durant les premières epochs. La perte passe d'environ 0.96 à une valeur proche de 0.72, ce qui montre que le modèle apprend progressivement à mieux représenter les données. Après cette première phase d'apprentissage rapide, la courbe devient plus stable et décroît plus lentement. Cela suggère que le réseau converge progressivement vers une solution relativement stable.

Le second graphique montre l'évolution de l'accuracy d'entraînement. Celle-ci augmente fortement dès les premières epochs, passant d'environ 52 % à plus de 60 % très rapidement. Ensuite, l'accuracy continue d'augmenter progressivement jusqu'à atteindre environ 67 %. L'accuracy d'entraînement est plutôt élevée pour un modèle aussi simple.

Concernant l'évaluation sur l'ensemble de test, le petit modèle a un résultat de 1.11 pour la perte et de 33.13 pour l'accuracy, ce qui est plutôt mauvais. Cela signale que le modèle n'a pas parvenu à généraliser convenablement. Ainsi, il faudrait augmenter la diversité des images et surtout leur qualité. Et tout simplement augmenter la complexité du modèle. En effet, les images sont floues et très bruitées, d'où la pertinence d'utiliser un VAE pour effectuer un débruitage des images ainsi que pour apprendre des caractéristiques des images.

Cependant, les deux courbes présentent plusieurs oscillations et irrégularités durant l'entraînement. Ces variations montrent que l'apprentissage reste relativement instable. Plusieurs facteurs peuvent expliquer ce comportement. Tout d'abord, le learning rate choisi peut être légèrement trop élevé, ce qui provoque des variations importantes des paramètres au cours de l'optimisation. Ensuite, la faible complexité du modèle limite sa capacité à apprendre des représentations robustes et stables.

Enfin, la qualité relativement faible des images du dataset constitue une difficulté importante pour l'apprentissage.

Afin de stabiliser davantage l'apprentissage, il serait possible de diminuer le learning rate ou d'utiliser des techniques supplémentaires de régularisation comme le dropout.

Concernant l'évaluation sur l'ensemble de test, malgré des performances d'entraînement relativement correctes, les résultats sur l'ensemble de test restent faibles. Le petit modèle a un résultat de 1.11 pour la perte et de 33.13 pour l'accuracy, ce qui est plutôt mauvais. Cela signale que le modèle ne parvient pas à généraliser convenablement. En effet, cette accuracy est proche d'une classification aléatoire dans un problème à trois classes équilibrées. Cette différence importante entre les performances d'entraînement et de test indique que le modèle apprend certaines caractéristiques spécifiques des données d'entraînement sans réussir à construire des représentations suffisamment robustes pour être réutilisées sur des données inconnues. Plusieurs éléments peuvent expliquer cette difficulté. Les images du dataset sont relativement bruitées, parfois floues et présentent peu de détails visibles. De plus, l'architecture utilisée reste très simple pour une tâche de classification d'images aussi complexe. Ces résultats justifient donc l'utilisation de modèles plus avancés dans la suite du projet. En particulier, l'utilisation d'un Variational Autoencoder (VAE) pour effectuer un débruitage des images ainsi que pour apprendre des caractéristiques des images.

4.3.2 Premier modèle avec l'aide d'un VAE

L'entraînement du VAE a mené à des résultats presque ininterprétables, et il semblerait que le modèle n'est pas atteint la qualité que nous désirions. Bien que pour l'entraînement d'un VAE, les critères d'évaluation ne sont pas les mêmes, a priori, avant même d'entamer une classification, l'évolution de la Validation loss et de la train loss suggère que le modèle n'a pas appris.

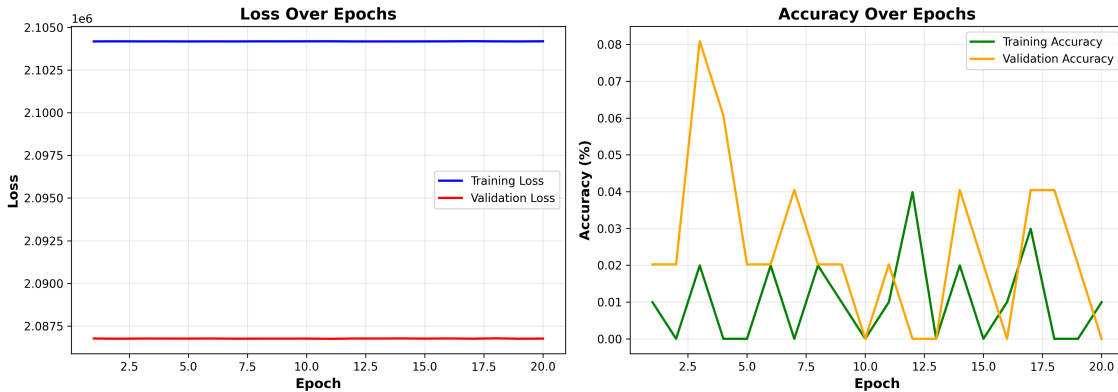


FIGURE 5 – Graphique, entraînement du premier VAE (modèle 2) - 20 epochs

Afin d'évaluer plus concrètement l'utilité de l'espace latent appris par le VAE, nous avons construit un modèle complémentaire appelé VAEClassifier. Ce modèle récupère directement les représentations latentes produites par l'encodeur du VAE puis ajoute un classificateur linéaire chargé de prédire les classes des galaxies. Le classificateur utilise les paramètres de l'espace latent, notamment la moyenne et la variance produites par l'encodeur, afin de construire une représentation exploitable pour la classification. Nous entraînons ce modèle sur 10 epochs afin d'avoir une idée claire de la pertinence de l'architecture du VAE précédent.

Sur les premières epochs, nous observons une diminution régulière de la training loss, qui passe progressivement d'environ 1.0995 à 1.096. Cette baisse reste relativement faible mais montre néan-

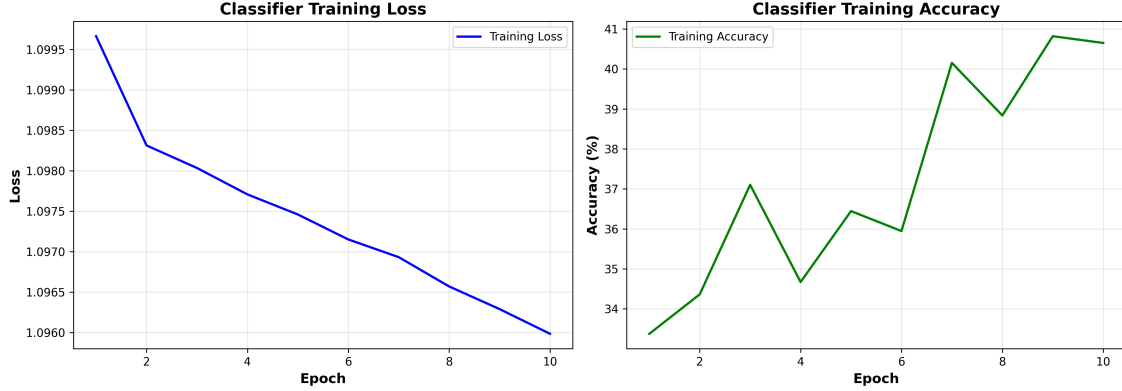


FIGURE 6 – Graphique, entraînement du classifieur s'appuyant sur le modèle 2 (modèle 2bis) - 10 epochs

moins que le modèle commence à apprendre certaines informations utiles. L'accuracy d'entraînement évolue également de manière encourageante. Elle passe d'environ 33 % au début de l'apprentissage à près de 41 % après seulement quelques epochs

Etant donné ces résultats plutôt encourageants puisque la loss semble bien diminuer, et l'instabilité de la reconstruction pour les premières epochs, nous augmentons le nombre d'epochs pour l'entraînement, en espérant une stabilisation sur le long terme (50 epochs).

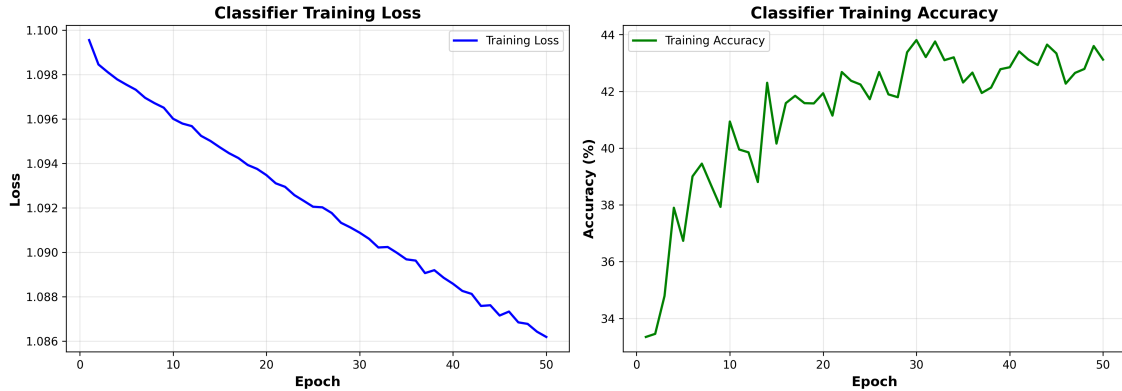


FIGURE 7 – Graphique, entraînement du classifieur s'appuyant sur le modèle 2 (modèle 2bis) - 50 epochs

Les graphiques montrent l'évolution de la loss et de l'accuracy sur cette phase d'entraînement plus longue. La training loss diminue cette fois de manière beaucoup plus régulière et stable, passant progressivement de 1.099 à environ 1.086. L'évolution de l'accuracy est également intéressante. Après plusieurs fluctuations durant les premières epochs, les performances finissent par se stabiliser autour de 43 à 44 %. Cela suggère que le modèle parvient à extraire certaines caractéristiques utiles des images grâce à l'espace latent appris par le VAE. Les oscillations observées au cours de l'entraînement restent cependant importantes, ce qui traduit une certaine instabilité du modèle et une difficulté à converger vers une représentation optimale. Les résultats de l'entraînement au bout de 50 epochs sont donc encourageants mais ils ne sont pas suffisants. Cela peut venir d'un manque de calibration du VAE, ou encore de la simplicité du modèle de classification choisi. Plutôt que de poursuivre dans cette direction, nous avons décidé de tester un nouveau modèle, et de revenir au VAE si les résultats

ne s'améliorent pas. L'accuracy pour le test étant de 44 %, cela reste mieux que le petit CNN mais toujours insuffisant (inférieur à 50 %).

4.3.3 Troisième modèle, Residual Network (ResNet-18)

Le troisième modèle testé dans notre projet est un réseau résiduel de type ResNet-18. Après les résultats relativement limités obtenus avec le petit CNN et le VAE, l'objectif est ici d'utiliser une architecture plus adaptée.

Dans leur papier [6], Han et al. entraînent un modèle ResNet-50 sur un très grand dataset pendant environ 50 epochs. Cependant, notre architecture étant moins profonde et notre dataset beaucoup plus réduit, nous avons choisi dans un premier temps un entraînement plus court de 20 epochs. Cette décision permet également de limiter le risque d'overfitting ainsi que le coût computationnel de l'entraînement, particulièrement important dans notre cas puisque les modèles sont entraînés uniquement sur CPU. Après ce premier essai sur 20 epochs, nous remarquons que le modèle overfit à partir de la cinquième epoch, donc nous décidons de faire du "early stopping", et d'entraîner seulement pour 4 epochs. Cette méthode consiste à interrompre l'entraînement avant la fin prévue lorsque les performances cessent de s'améliorer sur les données de validation. Les résultats sont très satisfaisants, les courbes ne sont pas linéaires du fait de la stabilisation du modèle, et la matrice permet de constater les résultats du modèle :

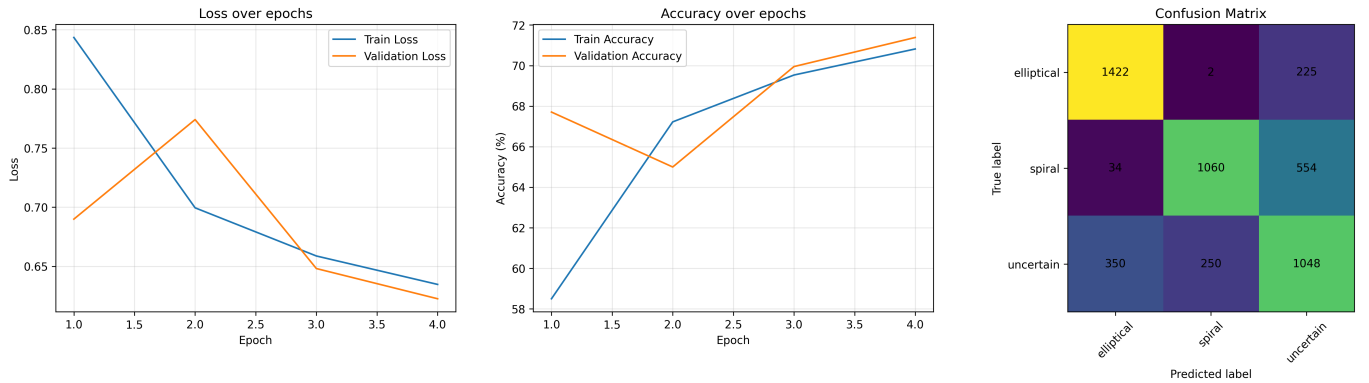


FIGURE 8 – Graphique, entraînement de ResNet-18 (modèle 3) - 4 epochs

La matrice de confusion obtenue confirme également les bonnes performances du modèle. Les galaxies spiral et elliptical sont largement mieux reconnues que dans les modèles précédents, et même la classe uncertain bénéficie d'une amélioration importante. Sur l'ensemble de test, le modèle atteint une accuracy de 71.39 % avec une loss de 0.6226. Ces résultats représentent une amélioration très importante par rapport aux autres architectures testées précédemment. Le gain de performance est particulièrement significatif si l'on compare ce résultat aux 33 % obtenus avec le petit CNN ou encore aux 44 % du VAEClassifier.

4.3.4 Quatrième modèle, Residual Network (ResNet-34)

Pour ce dernier modèle, étant satisfaites des résultats du réseau résiduel peu profond, nous avons ajouté des couches ainsi que prolongé l'entraînement (8 epochs à la place de 4). Cela nous a permis d'obtenir de meilleurs résultats. De plus, nous avons réalisé du fine-tuning assez important, qui consiste à baisser le learning rate quand le modèle semble converger afin que les paramètres soient

plus précis. Néanmoins, les résultats montrent que le modèle avec moins de couches parvenait à davantage généraliser. Sachant que notre dataset n'est pas extrêmement grand et sûrement moins grand que celui de l'article [6], ceci explique l'overfitting pour le modèle plus profond :

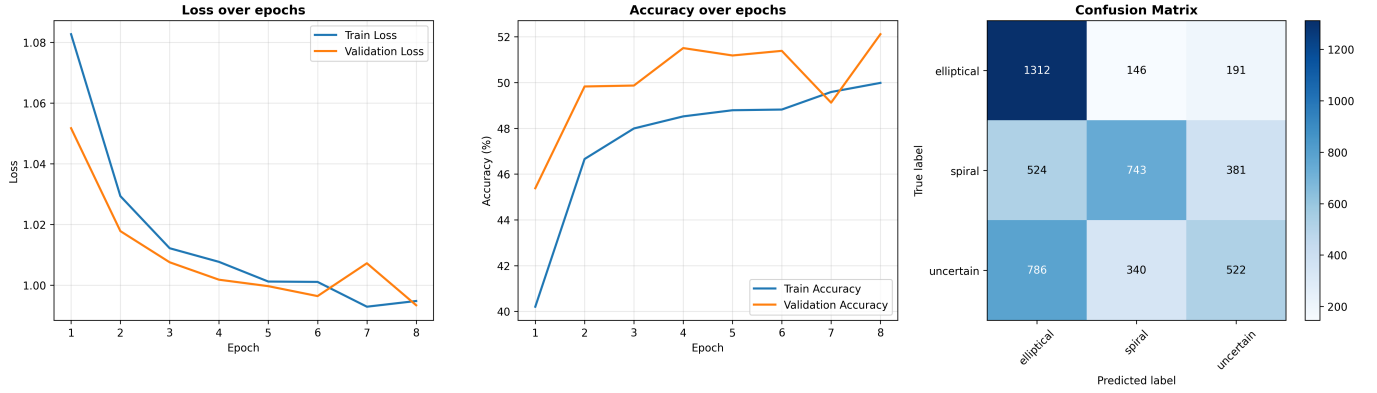


FIGURE 9 – Graphique, entraînement de ResNet-34 (modèle 4) - 8 epochs

Nous obtenons une accuracy de 52.11 % et une perte de 0.9934 sur le test, soit de moins bonnes capacités prédictives. Peut-être avec davantage de data augmentation ou encore de débruitage des images, donc avec un dataset plus grand, nous aurions pu fournir de meilleurs résultats avec un modèle aussi complexe que le ResNet-34. Cela peut également être dû à la calibration de l'apprentissage, et au fine-tuning qui a pu favoriser l'overfitting d'autant plus si l'ajustement du learning rate n'était pas suffisant et réduit en fonction de la convergence des paramètres du modèle. Nous remarquons également que l'early stopping dans ce cas n'est pas le problème principal, car l'accuracy de test converge vers 0.5, presque au niveau d'un modèle aléatoire (ce qui est un réel problème).

4.3.5 Synthèse

TABLE 6 – Comparaison des performances de tous les modèles

Modèle	Train Acc (%)	Test Acc (%)	Test Loss	Epochs
<i>Machine Learning</i>				
Decision Tree	95.43	55.79	—	—
Random Forest	—	64.90	—	—
<i>Deep Learning</i>				
Petit CNN (modèle 1)	≈70	33.13	1.1100	50
VAE + Classifieur (modèle 2)	≈44	44.00	—	50
ResNet-18 (modèle 3)	≈71	71.39	0.6226	4
ResNet-34 (modèle 4)	≈52	52.11	0.9934	8

5 Conclusion

Au cours de ce projet, nous avons étudié une tâche de classification d’images de galaxies en utilisant différentes approches de machine learning et de deep learning. Après la création et le pré-traitement d’un dataset équilibré d’environ 15 000 images, nous avons comparé plusieurs modèles afin d’identifier la méthode la plus adaptée à ce problème.

Les modèles de machine learning classiques, comme l’arbre de décision et la Random Forest, ont permis d’obtenir des premiers résultats intéressants, tout en mettant en évidence les limites de ces approches pour le traitement direct d’images complexes. Les modèles de deep learning ont ensuite montré des performances plus contrastées : le petit CNN et le VAE restent limités, tandis que le ResNet-18 obtient les meilleurs résultats avec une accuracy de 71.39 % sur l’ensemble de test. À l’inverse, le ResNet-34 montre qu’une architecture plus profonde ne garantit pas nécessairement de meilleures performances, notamment lorsque le dataset reste relativement limité et que les images sont bruitées. Dans le cadre de ce projet, le ResNet-18 apparaît comme le meilleur compromis entre performance, complexité et capacité de généralisation.

Ce projet nous a également permis de mieux comprendre l’importance du pré-traitement des données, du choix de l’architecture, de la gestion de l’overfitting ainsi que du réglage des hyperparamètres dans l’apprentissage profond. Les différences importantes observées entre les modèles montrent que la qualité des données et la capacité de généralisation sont des éléments centraux dans les tâches de classification d’images. De plus, nous comprenons mieux maintenant que les performances d’un modèle ne dépendent pas uniquement de sa complexité. Un modèle plus profond ou plus coûteux n’est pas toujours meilleur.

Aujourd’hui, les réseaux de neurones constituent des outils essentiels pour aider les chercheurs à étudier la structure et l’évolution de l’univers. En effet, les méthodes de classification automatique permettent aujourd’hui d’analyser des quantités extrêmement importantes de données astronomiques, ce qui serait impossible manuellement.

Références

- [1] In *Premier échantillon de données utilisé, collectées par des volontaires*. Galaxy Zoo, 2007.
- [2] In *Second échantillon de données utilisé, collectées par des volontaires (auteur : Willett et al.)*. Galaxy Zoo 2, 2013.
- [3] Anusha Guruprasad. Galaxy classification : A machine learning approach for classifying shapes using numerical data. 2023.
- [4] Ashley Spindler Leon H. Butterworth. The effects of image augmentations when training machine learning models in astronomy. 2026.
- [5] S. S. Mirzoyan. Enhancing galaxy classification with u-net variational autoencoders for image denoising. Juin 2025.
- [6] Liu Ling Zhang, Han. A multi-modal fusion network for star-galaxy classification from csst simulated datasets. Avril 2026.